Binary Code Similarity Detection via LLM-Based Source Code Conversion

Bangrui Wan, Shiyu Wang, Zheng Wei, Jiangping Huang, and Chunqiang Hu, *Member, IEEE*

***Abstract*—Binary code similarity detection (BCSD), a technique for assessing the similarity between two given binary code snippets, holds significant value in searching for vulnerable functions within embedded device firmware, which is typically closed-source. However, existing BCSD approaches face two major challenges: 1) the irreversible loss of semantic and structural information during the process of binary code compilation, which affects detection performance and 2) the inability to directly perform similarity detection between binary code and source code. In this article, we present Bin2SrcSim, a novel BCSD approach that employs a large language model (LLM) to convert binary code into source code representations. Bin2SrcSim fine-tunes an LLM at the function-level to transform assembly code and pseudocode into source code. Consequently, the similarity between any two binary code functions can be assessed by calculating the cosine similarity and Jaccard similarity of the transformed source code. The experimental results demonstrate that Bin2SrcSim outperforms all baselines, achieving Recall@1 scores of 0.82, 0.83, 0.93, and 0.81 across various scenarios involving cross-architecture, cross-compiler, and cross-optimization levels. Bin2SrcSim also demonstrates satisfactory performance in vulnerable function search within real-world Internet of Things device firmware. Moreover, Bin2SrcSim supports similarity detection between binary code and source code, expanding the scope of detection applications.**

***Index Terms*—Binary code similarity, large language model (LLM), source code conversion, vulnerable function search.**

I. Introduction

With the continuous development of foundational technologies such as smart terminals and mobile Internet, the Internet of Things (IoT) has rapidly emerged on a global scale. However, this growth has been accompanied by a frequent occurrence of IoT security incidents. The IoT security landscape report released by Netgear [1] highlights the significant security challenges faced by IoT embedded devices. Based on threat intelligence analysis from 2.6 million smart home samples worldwide, the report finds that devices in home networks are subjected to an average of eight attacks every 24 h. In practical IoT environments, IoT embedded devices often contain numerous known vulnerabilities. This is primarily due to the widespread use of open-source third-party components by IoT manufacturers to enhance development efficiency, inadvertently accelerating the spread of security vulnerabilities. Existing research [2] has indicated that many firmware updates for embedded devices include third-party components with known vulnerabilities for several years, and 80.4% of manufacturers have released firmware containing known vulnerabilities. Embedded devices from different manufacturers, or different models from the same manufacturer, may face similar security threats due to the use of a third-party component with security vulnerabilities.

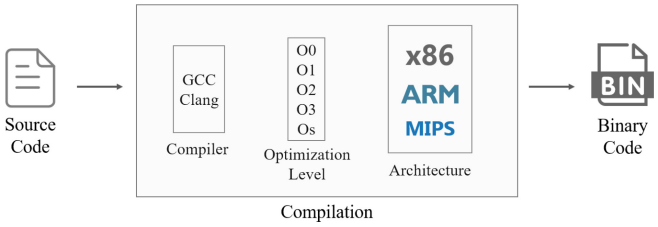

Fig. 1. Compilation process from source code to binary code. Variations in architecture, compiler, and optimization level can result in substantial differences in the binary code compiled from the same source code.

Binary code similarity detection (BCSD) is a technique for assessing the similarity between two given binary code snippets, typically at the function level, specifically to determine whether two given binary functions derive from the same source code. Given that firmware for embedded devices from IoT manufacturers is often closed-source, BCSD holds significant value in vulnerable function search within embedded devices. A common requirement is to detect whether a known vulnerable function exists within the firmware of an embedded device.

Embedded devices feature diverse CPU architectures at the hardware level, and the same firmware source code, when compiled for different architectures, results in significantly different binary code. As shown in Fig. 1, during the compilation process of embedded device firmware, the compiler (e.g., GCC

Received 17 April 2025; revised 24 May 2025; accepted 4 June 2025. Date of publication 12 June 2025; date of current version 8 December 2025. This work was supported in part by the Chongqing Technology Innovation and Application Development Special Major Project under Grant CSTB2023TIAD-STX0034; in part by the National Natural Science Foundation of China under Grant 62372075; in part by the Natural Science Foundation of Chongqing, China, under Grant CSTB2024NSCQ-LZX0084; and in part by the Humanities and Social Sciences Fund of the Ministry of Education under Grant 20YJCZH047. *(Corresponding author: Chunqiang Hu.)*

Bangrui Wan is with the School of Big Data and Software Engineering, Chongqing University, Chongqing 401331, China, and also with the School of Software Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: wanbr@cqupt.edu.cn).

Shiyu Wang, Zheng Wei, and Jiangping Huang are with the School of Software Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: sy.wang.dev@gmail.com; weizheng171514@gmail.com; huangjp@cqupt.edu.cn).

Chunqiang Hu is with the School of Big Data and Software Engineering, Chongqing University, Chongqing 401331, China (e-mail: chu@cqu.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3579231

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

and Clang), the optimization level (e.g., O0, O1, O2, O3, and Os) and the target architecture (e.g., x86, ARM, and MIPS) must be explicitly specified to generate the corresponding binary code. However, variations in architecture, compiler, and optimization level can lead to substantial differences in the binary code compiled from the same source code [3], [4], [5], posing significant challenges for BCSD.

Existing learning-based BCSD approaches typically follow three key phases: 1) binary code representation; 2) feature learning; and 3) similarity comparison. First, since binary code is essentially composed of 0s and 1s, it needs to be transformed into a higher-level abstraction to capture its semantic and structural information. To this end, existing approaches [6], [7] commonly employ disassemblers to convert binary code into assembly code and then extract basic blocks, control flow graph (CFG), data flow graph (DFG), and attributed CFG from it. Second, based on the binary code representation, deep learning models are employed for feature learning. some approaches [8], [9] utilize graph embedding techniques, inputting CFG or DFG into graph neural networks (GNNs) to learn features of nodes and edges and generate embedded representations of the graphs. Other approaches [6], [7], [10] adopt code embedding techniques, treating code as a sequence of instructions and employing natural language processing (NLP) techniques to conduct in-depth semantic feature learning on instruction sequences. Finally, based on the extracted features from the previous step, the similarity of code is assessed by calculating the distance between feature vectors or introducing a learning-based mapping function [11], [12].

Although existing approaches have made certain progress in BCSD, there is still room for improvement. First, the nature of binary code leads to an irreversible loss of semantic and structural information, which significantly restricts the performance of BCSD. Since the compilation process entails substantial loss of high-level semantic information during the compilation of source code, the assembly instructions and CFG obtained through disassembly inevitably suffer from loss of semantic and structural information, which is a key factor affecting the effectiveness of BCSD. Some approaches [13], [14], [15] convert binary code into a unified intermediate representation (IR) for similarity detection, primarily aiming to enhance the semantic and structural information of binary code and reduce the differences caused by different architectures and compilers.

Second, existing approaches are incapable of directly performing similarity detection between binary code and source code. Current approaches typically require binary code as input, which poses limitations in certain specific application scenarios. For instance, when it is necessary to determine whether a vulnerable source code function exists within the firmware of an embedded device, and the source code function cannot be compiled, existing approaches fail to work effectively, thereby restricting their scope of application.

Inspired by the powerful code capabilities of current large language models (LLMs) [16], [17], [18], we propose a novel BCSD approach named Bin2SrcSim, which employs LLM to convert binary code into source code representations for similarity detection. Fig. 2 illustrates the core detection workflow of Bin2SrcSim. Initially, we utilize a decompiler to obtain the assembly code and pseudocode of the binary code at the function-level. Subsequently, these representations are fed into a fine-tuned LLM to convert the binary code into source code form. During this process, the fine-tuned LLM supplements and enhances the high-level semantic and structural information lost during the compilation process. Ultimately, by calculating cosine similarity and Jaccard similarity, the similarity between any two binary code functions can be assessed.

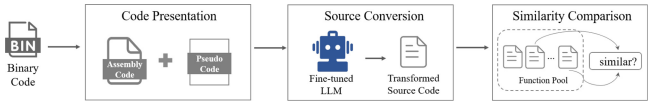

Fig. 2. Core detection workflow of Bin2SrcSim (this work). It comprises three steps: code representation, source conversion, and similarity comparison. Binary code from different compilers, optimization levels, and architectures can be converted into source code form based on the fine-tuned LLM for similarity detection.

We implemented a prototype of Bin2SrcSim and evaluated it on diverse datasets. The results demonstrate that Bin2SrcSim outperforms all baseline approaches, including Gemini [19], SAFE [10], and Asteria-Pro [20], as well as the unfine-tuned LLMs GPT-4o [16], CodeLlama [17], and Qwen [18]. Specifically, Bin2SrcSim achieves Recall@1 scores of 0.82, 0.83, 0.93, and 0.81 across various scenarios involving cross-architecture, cross-compiler, and cross-optimization levels. In the vulnerable function search within real-world IoT device firmware, Bin2SrcSim also demonstrates satisfactory performance. Moreover, Bin2SrcSim supports similarity detection between binary code and source code, thereby expanding the scope of BCSD applications.

In summary, the key contributions of this article are as follows.

1) We propose a novel LLM-based approach for BCSD, named Bin2SrcSim. To the best of our knowledge, this is the first work that performs BCSD by converting binary code into source code representations based on LLM. The code for Bin2SrcSim is publicly available on GitHub at https://github.com/SQAbin/Bin2SrcSim.
2) We conduct extensive experiments, demonstrating that Bin2SrcSim outperforms all baseline approaches. Specifically, Bin2SrcSim is capable of performing similarity detection between binary code and source code, expanding the scope of BCSD applications.
3) We validate the effectiveness of Bin2SrcSim across different LLMs and investigate the impact of various combinations of assembly code, CFG, and pseudocode on BCSD detection performance, laying the foundation for subsequent related research.

II. Related Work

BCSD has consistently been a research hotspot. This section reviews the related work in two key areas: 1) binary code representation and 2) feature extraction.

A. Binary Code Representation

Binary code, in its raw form, consists of machine-level instructions that are difficult to analyze directly. To facilitate similarity detection, various representations that capture the

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

semantic and structural information of binary code have been proposed in relevant studies.

Assembly language, a low-level programming language, utilizes mnemonics and operands to represent machine instructions. Disassembly tools convert binary code into assembly instructions, which are subsequently employed for feature extraction. For example, Asm2Vec [6] learns semantic relationships between assembly instructions and represents functions as vectors, enabling robust similarity detection despite code obfuscation and compiler optimizations. BinDeep [7] employs a siamese neural network combining CNN and LSTM to vectorize instruction sequences, effectively handling cross-architecture and cross-compiler scenarios.

CFG represents programs as directed graphs, where nodes denote basic blocks and edges represent control flow transitions. This representation captures structural information and execution paths, making it suitable for cross-compiler and cross-optimization comparisons. BSGAT [21] extracts CFG and uses random walks to generate instruction sequences, which are then processed by a graph attention network to produce function embeddings. Other approaches, such as Gemini [19], enhance CFG by attaching attribute information (e.g., constants and instruction counts) to form attributed CFG. DeepBinDiff [22] combines CFG with call graphs to create interprocedural CFG, providing program-wide context. IoTSim [23] generates data CFGs by incorporating data dependencies and address adjacency relationships.

IR provide a unified representation of binary code, reducing the impact of architecture, compiler, and optimization differences. Common IR include LLVM IR, microcode, and pseudocode. Oscar [13] converts binary code into LLVM IR and uses a hierarchical Transformer encoder for feature extraction. VulHawk [14] and IoTSim [23] employ microcode as an IR, leveraging pretrained language models to generate embeddings. Pseudocode, which resembles high-level language constructs, offers a more abstract representation. UPPC [15] uses a deep pyramid convolutional neural network to embed pseudocode and strings, enabling effective similarity detection.

B. Feature Extraction

Graph embedding approaches represent binary code as graphs (e.g., CFG and DFG) and use GNNs to learn node and edge features. Gemini [19] employs Structure2vec to embed CFGs and computes similarity using a siamese network. BVSNO [24] uses BLSTM and CNN to extract the features in PDG and aggregate them for detection. HermesSim [9] processes semantic-oriented graphs with GNNs and introduces a multihead softmax aggregator to fuse information.

Code embedding approaches treat binary code as instruction sequences and apply NLP techniques to extract semantic features. Asm2Vec [6] trains neural networks to learn assembly code embeddings, while BinDeep [7] uses Word2Vec to vectorize instruction sequences and combines CNN and LSTM for feature extraction. SAFE [10] employs self-attentive neural networks to generate function embeddings from disassembled code.

Recent approaches leverage pretrained models to transform binary code into vector representations. jTrans [25] enhances Transformer models with control flow information through jump target prediction tasks. Codeformer [26] uses multihead attention to embed basic blocks, while UniASM [27] integrates token embeddings, positional embeddings, and segment embeddings via a Transformer architecture.

Hybrid approaches combine graph and code embeddings to leverage both structural and semantic information. Codee [28] generates basic block embeddings using CFG and token embeddings using NLP techniques, then applies tensor singular value decomposition for function embedding. BSGAT [21] integrates graph attention networks with NLP to detect binary function similarity. HGE-BVHD [29] maps architecture-specific embeddings to a shared space via adversarial training and uses relational GCNs to aggregate node features.

III. Methodology

A. Overview

We propose an approach for BCSD named Bin2SrcSim, which leverages an LLM to transform binary code into source code representations for the purpose of similarity detection. Fig. 3 illustrates the overall architecture of Bin2SrcSim, which encompasses three primary stages, namely the binary-source matching stage, the LLM fine-tuning stage, and the similarity detection stage.

In the binary-source matching stage, based on several popular open-source components, we match the binary code of each function with its corresponding source code. This matching process aims to prepare for the generation of fine-tuning instruction set for the subsequent LLM fine-tuning stage. (Section III-B).

In the LLM fine-tuning stage, we carefully select a part of binary code functions along with their corresponding source code and combine them to form the fine-tuning instruction set. Subsequently, this set is used to fine-tune the LLM, enabling it to take assembly code and pseudocode as inputs and generate the corresponding source code as output. (Section III-C).

In the similarity detection stage, binary code from various architectures, compilers, and optimization levels can be transformed into source code with the aid of the fine-tuned LLM. Subsequently, the similarity between any two binary code functions is compared by calculating cosine similarity and Jaccard similarity. (Section III-D).

B. Binary-Source Matching

In this stage, leveraging several popular open-source components (detailed in Dataset1, including Curl, ClamAV, Nmap, Unrar, and Zlib), we match the binary code of each function with its corresponding source code.

Formally, let F denote the set of binary code functions contained within a binary code file, with corresponding sets for assembly code A, pseudocode P, and source code S. Specifically, we consider the compiler set $C = \{\text{GCC, Clang}\}$, optimization level set $O = \{\text{O0, O1, O2, O3, Os}\}$, and target machine architecture set $M = \{\text{x86, x64, ARM32, ARM64, MIPS32}\}$. The binary-source matching stage encompasses four

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

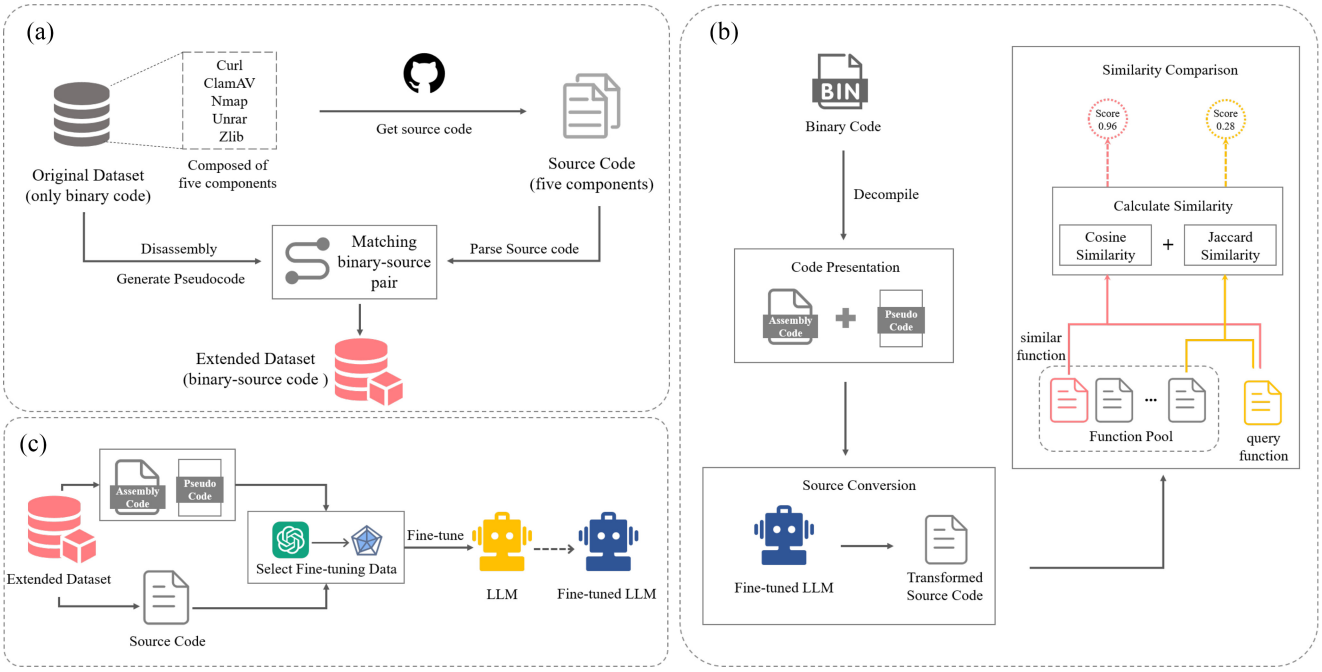

Fig. 3. Overview of the Bin2SrcSim framework. (a) Binary-source matching stage aims to prepare for the generation of fine-tuning instruction set. (b) LLM fine-tuning stage involves fine-tuning the LLM to enable it to take assembly code and pseudocode as inputs and generate the corresponding source code as output. (c) Similarity detection stage assesses the similarity between any two binary code functions.

steps, such as disassembly, pseudocode generation, source code parsing, and binary-source matching.

1) Disassembly: Assembly code, as a direct representation of machine code, closely approximates the hardware level and reveals the low-level execution logic of software programs. Assembly code enables us to parse the operation instructions and their execution order in binary code. Given that binary code consists essentially of 0s and 1s and is difficult to analyze directly, the first step in BCSD typically involves using disassembly techniques to convert binary code into assembly code. We employ RetDec [30] as the decompiler for disassembly. By identifying control flow breakpoints in the assembly code (e.g., function calls and return statements), we can determine the boundaries of each function and thereby extract the assembly code corresponding to each binary code function, denoted as

$$\{a_i\}_{i=1}^{n} = \mathrm{Func}_{\mathrm{disasm}}(\mathrm{Binaryfiles}) \tag{1}$$

where $a_i \in A$, Binaryfiles denotes the set of binary code files to be disassembled, and n represents the total number of binary code functions obtained from the disassembly of these binary files.

2) Pseudocode Generation: Pseudocode, a C-like IR, can also be generated using RetDec. Although the resulting pseudocode may not always be precise, it can still mitigate the complexity of understanding assembly code. Given assembly code, the corresponding pseudocode can be obtained as follows:

$$\{p_i\}_{i=1}^{n} = \mathrm{Func}_{\mathrm{psd}}\left(\{a_i\}_{i=1}^{n}\right) \tag{2}$$

where $p_i \in P$. Through the preceding steps, each binary code file has been decomposed into pairs of assembly code and pseudocode at the function level. Considering the distinct combinations of compilers, optimization levels, and architectures, the binary code of each component manifests in $|C| \times |O| \times |M|$ forms, hence

$$\left\{f_i^{(c_i,o_i,m_i)}\right\}_{i=1}^{n} = \{a_i, p_i\}_{i=1}^{n} \tag{3}$$

where $c_i \in C$, $o_i \in O$, $m_i \in M$, and $f_i^{(c_i,o_i,m_i)} \in F$.

3) Source Code Parsing: In this step, we obtain the source code files corresponding to the binary code files from the open-source community and parse them by function name to extract each source code function

$$\{s_i\}_{i=1}^{u} = \mathrm{Func}_{\mathrm{src}}(\mathrm{SourceFiles}) \tag{4}$$

where $s_i \in S$, SourceFiles denotes the set of source code files corresponding to the binary code files, and u represents the total number of source code functions parsed from these source code files.

4) Binary-Source Matching: In this step, each binary code function $f_i^{(c_i,o_i,m_i)}$ is matched with its corresponding source code function s_i. This matching is performed based on the combination of component name and function name. Specifically, during the disassembly process, we extract each binary code function's name. Subsequently, each function is examined to determine whether a source code function with the same name exists within the same component of the source code set. If such a function exists, the matching is deemed successful; otherwise, it fails, and the binary code function $f_i^{(c_i,o_i,m_i)}$ is removed from the binary code function set F. For the successfully matched binary code functions and source code functions, there exists a unique mapping relationship R, such that each binary code function $f_i^{(c_i,o_i,m_i)}$

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EVALUATION CRITERIA FOR FINE-TUNING DATA

Dimension	Score Range	Objective
Semantic Correspondence	0-40	The accuracy with which the source code captures the functionality of the assembly code.
Code Structure	0-20	Clarity of logical code organization and appropriateness of function signature design, as well as the structured nature of the implementation.
Implementation Quality	0-20	Elegance and optimization of the code.
Completeness	0-10	Inclusion of all necessary operations from the assembly code.
Self-containment	0-10	The ability of the code to run independently without relying on external function calls.

can be mapped to its corresponding source code function s_i through R, expressed as

$$\forall f_i^{(c_i,o_i,m_i)} \in F, \exists! s_i \in S | \left(f_i^{(c_i,o_i,m_i)}, s_i\right) \in R. \tag{5}$$

Ultimately, we obtained the assembly code, pseudocode, and matched source code for each binary code function.

C. LLM Fine-Tuning

In this stage, we aim to train an LLM capable of converting binary code functions into source code. Specifically, the LLM takes assembly code and pseudocode as inputs and generates the corresponding source code as output. The transformed source code, enhanced with semantic and structural information by the LLM, contributes to improving the effectiveness of subsequent code similarity detection. This stage comprises two steps: 1) fine-tuning data selection and 2) fine-tuning LLM.

1) Fine-Tuning Data Selection: Given that LLMs lack proprietary training data for the specific task of converting binary code functions into source code, direct application of the LLM cannot ensure the effectiveness of the conversion. Therefore, the primary task of fine-tuning is to select appropriate fine-tuning data. However, it is evident that the amount of data used for LLM fine-tuning is significantly smaller than the corpus used during pretraining. Thus, selecting high-quality fine-tuning data is crucial. The main objective of fine-tuning data selection is to enhance the LLM's performance on the specific task while maintaining data efficiency and integrity. Inspired by existing research [31], we have designed five dimensions of fine-tuning data evaluation criteria: 1) semantic correspondence; 2) code structure; 3) implementation quality; 4) completeness; and 5) self-containment, as detailed in Table I.

We provide the assembly code, pseudocode, and matched source code for each binary code function to an LLM (e.g., GPT-4o), which scores them based on the five dimensions and selects those scoring above a predefined threshold. To minimize the randomness and uncertainty of the LLM's output, we set the temperature value of the LLM to 0. Meanwhile, we ensure that the fine-tuning data cover various compilers, optimization levels, and architectures, with the count for each combination not deviating too much from $|C| \times |O| \times |M| = 50$. Additionally, we established a sample quantity threshold k to balance the fine-tuning effect and efficiency of the LLM, ensuring that the number of samples does not exceed k, with k being in the thousands. Based on the above design, we obtain

$$\left(\left\{f_j^{(c_j,o_j,m_j)}\right\}_{j=1}^{k}, \{s_j\}_{j=1}^{k}\right) = \mathrm{Func}_{\mathrm{ftds}}\left(\left\{f_i^{(c_i,o_i,m_i)}\right\}_{i=1}^{n}, \{s_i\}_{i=1}^{n}\right) \tag{6}$$

where $\left(f_i^{(c_i,o_i,m_i)}, s_i\right) \in R$ and $\left(f_j^{(c_j,o_j,m_j)}, s_j\right) \in R$. Thus, the selected $\left(\{f_j^{(c_j,o_j,m_j)}\}_{j=1}^{k}, \{s_j\}_{j=1}^{k}\right)$ form the fine-tuning instruction set. Furthermore, we conduct manual sampling checks on the fine-tuning instruction set to ensure compliance with the above requirements.

2) Fine-Tuning LLM: From the perspective of parameter scale, there are two technical approaches for fine-tuning LLMs: one is to train all parameters of the model, known as full fine-tuning; the other is to train only a subset of parameters, known as efficient fine-tuning. Given that the parameter count of current LLMs is at least in the hundreds of millions, employing efficient fine-tuning methods can significantly reduce training costs while maintaining performance.

During the adaptation to specific tasks, existing studies [32], [33] have indicated that the weights of pretrained models maintain full rank during pretraining but exhibit lower-rank characteristics when transitioning to new tasks. Inspired by this, we employ low-rank adaptation (LoRA) [34] for fine-tuning. The core concept of LoRA is that low-rank dimensions imply that data can be effectively approximated in a lower-dimensional space while retaining most of the critical information or structure.

Formally, to fine-tune the weight matrix W of the LLM, two low-rank matrices, denoted as W_A and W_B, are introduced such that the weight update ΔW can be expressed as

$$W + \Delta W = W + W_B W_A \tag{7}$$

where $W \in \mathbb{R}^{d\times d}$, and d represents the input and output dimensions of the multihead self-attention mechanism in the Transformer architecture of the LLM. $W_B \in \mathbb{R}^{d\times r}$ and $W_A \in \mathbb{R}^{r\times d}$, r is the rank set during fine-tuning, which is significantly smaller than d. The weight matrix W_B is initialized as a zero matrix, whereas the weight matrix W_A is initialized using a random Gaussian distribution. During the fine-tuning process, the weight matrix W is frozen and does not undergo gradient updates to maintain its parameters unchanged. In contrast, W_A and W_B, which contain trainable parameters, participate in the LLM's optimization process.

Specifically, the fine-tuning instruction set composed of the input vector x and the output vector h is utilized for fine-tuning the LLM, with the fine-tuning process defined as follows:

$$h = Wx + \Delta Wx = Wx + W_B W_A x \tag{8}$$

where $F_{\mathrm{finetune}}: \left(\left\{f_j^{(c_j,o_j,m_j)}\right\}_{j=1}^{k}, \{s_j\}_{j=1}^{k}\right) \rightarrow \{x_j\}_{j=1}^{k}, \{h_j\}_{j=1}^{k}$. $\left(f_j^{(c_j,o_j,m_j)}, s_j\right) \in R$, and F_{finetune} represents the conversion of

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

the input textual fine-tuning instruction set into a vector fine-tuning instruction set by the fine-tuning framework. In this study, LLaMA-Factory [35] is employed as the fine-tuning framework.

D. Similarity Detection

In this stage, binary code from different compilers, optimization levels and architectures can be converted into source code form based on the fine-tuned LLM for subsequent similarity detection. The stage consists of three steps: 1) code presentation; 2) source conversion; and 3) similarity comparison.

1) Code Presentation: For the binary code to be detected, similar to Section III-B, the assembly code and pseudocode of the binary code functions are extracted using the decompiler, and these extracted elements are used to form binary code functions $f_i^{(c_i,o_i,m_i)}$.

2) Source Conversion: The fine-tuned LLM is capable of converting the assembly code and pseudocode of each binary code function into source code form. Based on this, the transformed source code is represented as

$$\{\mathrm{ST}_i\}_{i=1}^{l} = \mathrm{Func}_{\mathrm{trans}}\left(\left\{f_i^{(c_i,o_i,m_i)}\right\}_{i=1}^{l}\right) \tag{9}$$

where l denotes the total number of binary code functions to be detected.

It should be noted that when detecting the similarity between binary code and source code, the source code does not require the source conversion step, and similarly, the code presentation step can also be omitted.

3) Similarity Comparison: Following the preliminary steps, the binary code functions to be detected have been converted into source code form. In practical detection scenarios, the binary code functions to be detected are typically divided into two parts: the query, which is usually a single binary code function; and the target, which is sourced from a pool containing multiple binary code functions. The objective of BCSD is to identify target functions in the pool that are similar to the query.

We employ a straightforward and efficient method to calculate both cosine similarity and Jaccard similarity, thereby determining the similarity score between two binary code functions. For cosine similarity calculation, we utilize the bag-of-words model to treat the transformed source code as plain text. The process involves tokenization, followed by the statistical analysis of the frequency of each token, and subsequently transforming the code into a vector form to calculate similarity. For Jaccard similarity calculation, we extract key elements such as function names, parameters, operators, and return values from the transformed source code to form sets, and then calculate the Jaccard similarity between these sets. We refrain from employing-learning-based methods to conduct semantic and structural learning on the transformed source code, primarily for two reasons: to ensure the simplicity and efficiency of similarity calculation, and to assess the effectiveness of LLM-based binary code into source code conversion.

First, the cosine similarity is calculated. For any two transformed source codes ST_i and ST_j, they are mapped to function vectors v_{ST_i} and v_{ST_j} by extracting lexical units, with the calculation formula being

$$\mathrm{Sim}_{\cos}(\mathrm{ST}_i, \mathrm{ST}_j) = \frac{v_{\mathrm{ST}_i} \cdot v_{\mathrm{ST}_j}}{\left\|v_{\mathrm{ST}_i}\right\| \times \left\|v_{\mathrm{ST}_j}\right\|}, i \neq j. \tag{10}$$

Subsequently, the Jaccard similarity is calculated. For the ST_i and ST_j, we extract their key elements to form sets b_{ST_i} and b_{ST_j}. The calculation formula is

$$\mathrm{Sim}_{\mathrm{jac}}(\mathrm{ST}_i, \mathrm{ST}_j) = \frac{\left|b_{\mathrm{ST}_i} \cap b_{\mathrm{ST}_j}\right|}{\left|b_{\mathrm{ST}_i} \cup b_{\mathrm{ST}_j}\right|}, i \neq j. \tag{11}$$

We combine cosine similarity and Jaccard similarity to measure the similarity between two binary code functions, with the formula given by

$$\mathrm{Score}(\mathrm{ST}_i, \mathrm{ST}_j) = \frac{1}{2}\left(\frac{1}{2}\left(\mathrm{Sim}_{\cos}(ST_i, ST_j) + 1\right) + \mathrm{Sim}_{\mathrm{jac}}(ST_i, ST_j)\right), \quad i \neq j. \tag{12}$$

The value range of Score is $[0, 1]$. A value closer to 1 indicates greater similarity between the ST_i and ST_j, whereas a value closer to 0 suggests greater dissimilarity.

IV. Experimental Evaluation

In this section, we evaluate Bin2SrcSim in order to answer the following research questions.

1) How does Bin2SrcSim perform across cross-compiler, cross-optimization, and cross-architecture scenarios? (Section IV-B).
2) How effective is Bin2SrcSim in real-world vulnerable function search? (Section IV-C).
3) How does Bin2SrcSim perform in detecting similarity between binary code and source code? (Section IV-D).
4) What is the impact of the capabilities of the LLM on Bin2SrcSim? (Section IV-E).
5) What is the impact on the performance of Bin2SrcSim when different representations are used to fine-tune the LLM? (Section IV-F).

A. Experiment Setup

1) Dataset: We designed a dataset for training the baseline approaches, as well as two datasets for evaluating the performance of cross-scenario detection and vulnerable function search, respectively.

Dataset1: This dataset is used to train the baseline models. It includes five popular open-source components: Curl 7.67.0, ClamAV 0.102.0-beta, Nmap 7.80, Unrar 5.5.3, and Zlib 1.2.11. These components are compiled using two compilers (GCC and Clang), five optimization levels (O0, O1, O2, O3, and Os), and five machine architectures (x86, x64, ARM32, ARM64, and MIPS32). Dataset1 is exclusively used for training the baseline models and fine-tuning Bin2SrcSim.

Dataset2: This dataset is designed to evaluate the performance of BCSD across different architectures, compilers, and optimization levels. It comprises five open-source

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

components commonly used in IoT devices: OpenSSL 3.0.0-alpha1, Libmicrohttpd 0.9.71, LibTomCrypt 1.18.2, Coreutils 8.32, and SQLite 3.34.0. These components are compiled using the same two compilers, five optimization levels, and five machine architectures as in Dataset 1. Additionally, it should be noted that the components in Dataset2 are different from those in Dataset1.

Dataset3: This dataset is used to evaluate the performance of vulnerable function search in real-world scenarios. It is a publicly available dataset [36] that includes firmware images from two IoT embedded devices: 1) Netgear R7000 (ARM32) and 2) TP-Link Deco M4 (MIPS32). The dataset also contains ten vulnerable functions from OpenSSL 1.0.2d, covering a total of eight CVEs. The libcrypto libraries in both firmware versions include these vulnerable functions.

2) Baselines: We select several approaches in BCSD as baselines.

Gemini [19]: It employs neural networks to compute embeddings based on the numerical vectors of the CFG of each binary function and similarity detection is achieved by measuring the distance between the embeddings of two functions.

SAFE [10]: It employs word2vec-based technology combined with self-attentive network to directly generate embeddings of binary functions from disassembled code. This approach efficiently processes binary instructions and extracts semantic information without the need for manual feature extraction.

Asteria-Pro [20]: It introduces domain-specific knowledge and utilizes prefiltering and reordering modules. It first filters out dissimilar functions using lightweight features and then reorders the candidate functions based on call relationships.

3) LLMs: In addition, we introduce a proprietary LLM, GPT-4o [16], and two open-source LLMs, CodeLlama [17] and Qwen (version 2.5) [18], for performance comparison. Among them, CodeLlama and Qwen can serve as the LLM within Bin2SrcSim for fine-tuning.

4) Metrics: In the experiments, we primarily utilize two metrics to evaluate performance: Recall@K and MRR.

Recall@K: Given a query function and a pool of binary code functions containing multiple candidate functions, Recall@k represents the proportion of the number of times the target function is successfully retrieved within the top K candidates to the total number of query functions, after sorting the candidate functions in descending order of similarity scores. The formula for calculating Recall@K is as follows:

$$\text{Recall@}K = \frac{1}{|Q|}\sum_{q\in Q}\mathbb{I}\big(\text{Rank}_q \le K\big) \tag{13}$$

where Q denotes the set of query functions, q represents an individual query function, and Rank_q indicates the position of the target function in the sorted list of candidate functions. $\mathbb{I}(\text{Rank}_q \le K)$ is an indicator function that equals 1 if $\text{Rank}_q \le K$ and 0 otherwise.

A higher Recall@K indicates that the approach is more likely to retrieve similar binary code functions within the top K results, reflecting its reliability in practical applications.

TABLE II
PERFORMANCE OF DIFFERENT APPROACHES

Models	Recall@1				MRR			
	XA	XC	XO	XM	XA	XC	XO	XM
Gemini	0.66	0.57	0.58	0.42	0.75	0.67	0.67	0.55
SAFE	0.23	0.21	0.21	0.20	0.37	0.39	0.38	0.35
Asteria-pro	0.62	0.42	0.57	0.37	0.68	0.50	0.62	0.44
CodeLlama-13B	0.11	0.11	0.17	0.09	0.16	0.15	0.24	0.13
Qwen-14B	0.13	0.14	0.25	0.11	0.21	0.22	0.34	0.18
GPT-4o	0.48	0.53	0.58	0.32	0.55	0.61	0.71	0.42
Bin2SrcSim	**0.82**	**0.83**	**0.93**	**0.81**	**0.86**	**0.89**	**0.95**	**0.87**

MRR: It measures the mean reciprocal rank of the correctly matched target function in the ranked results and serves as a relative score. The formula for calculating MRR is as follows:

$$\text{MRR} = \frac{1}{|Q|}\sum_{q\in Q}\left(\frac{1}{\text{Rank}_q}\right). \tag{14}$$

Compared to Recall@K, MRR not only examines whether similar binary code functions are retrieved within the top K results but also focuses on the specific ranking positions of these functions.

5) Implementation Details: We implemented a prototype of Bin2SrcSim using Python and PyTorch, adopting LLaMA-Factory as the fine-tuning framework. The server used for experiments is equipped with Ubuntu 22.04, standardized libraries, and consistent hardware configurations (dual 20-core CPUs, 384GB RAM, and three NVIDIA 4090 GPUs).

We trained Gemini and SAFE on Dataset1, while Asteria-pro used its pretrained model directly. We performed fine-tuning on Dataset1 for Bin2SrcSim using Qwen-14B. During the fine-tuning data selection, we set the fine-tuning data selection score threshold at 90 and the sample quantity threshold k at 1500.

We designed four distinct experimental scenarios: 1) *XA:* the function pairs have different architectures but the same compiler and optimization level; 2) *XC:* the function pairs have different compilers but the same architecture and optimization level; 3) *XO:* the function pairs have different optimization levels but the same compiler and architecture; and 4) *XM:* the function pairs are from arbitrary combinations of compilers, optimization levels, and architectures.

B. Cross-Scenario Performance

In this section, we aim to evaluate the performance of Bin2SrcSim across various scenarios, including cross-compiler, cross-optimization, and cross-architecture. The evaluation involves comparisons with baselines as well as both open-source and proprietary LLMs. It is important to note that none of these LLMs have undergone fine-tuning; instead, they directly process assembly code and pseudocode to generate similarity scores.

As illustrated in Table II, Bin2SrcSim consistently outperforms all baseline methods across cross-compiler, cross-optimization, and cross-architecture scenarios. Specifically, in terms of Recall@1 metric, Bin2SrcSim achieved scores

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

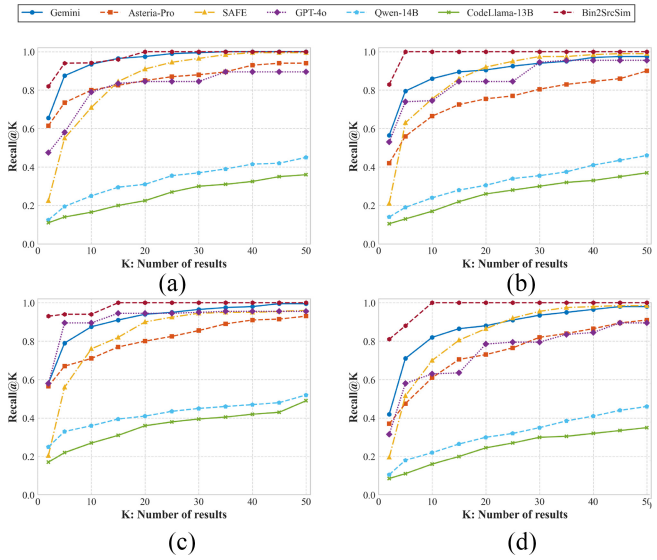

Fig. 4. Variation curve of Recall@K for different approaches. (a) XA. (b) XC. (c) XO. (d) XM.

of 0.82, 0.83, 0.93, and 0.81 in the XA, XC, XO, and XM scenarios, respectively. Compared to the best-performing baseline approach, Gemini, Bin2SrcSim demonstrates improvements of 16%, 26%, 35%, and 39% in these scenarios, respectively. These results indicate that Bin2SrcSim can more accurately recall similar binary code functions. Regarding the MRR metric, Bin2SrcSim also exhibits superior performance, achieving scores of 0.86, 0.89, 0.95, and 0.87 in the respective scenarios. Compared to Gemini, Bin2SrcSim achieves enhancements of 0.11, 0.22, 0.28, and 0.32 in the XA, XO, XC, and XM scenarios, respectively. These findings demonstrate that Bin2SrcSim not only detects similar binary code functions but also ranks them higher in the retrieval results, thereby enhancing the overall quality of the search outcomes.

Furthermore, the results indicate that the three LLMs exhibit suboptimal performance when directly employed for BCSD without fine-tuning. Specifically, CodeLlama-13B and Qwen-14B achieve Recall@1 scores ranging from 0.11 to 0.25 across different scenarios. Although GPT-4o outperforms CodeLlama-13B and Qwen-14B, its Recall@1 scores remain 34%, 30%, 35%, and 49% lower than those of Bin2SrcSim in the XA, XO, XC, and XM scenarios, respectively. This suggests that LLMs in an unfine-tuned state, struggle to effectively assess similarity based solely on assembly code and pseudocode.

In some practical BCSD tasks, such as vulnerability search, the K value in the Recall@K metric is often not limited to 1. Therefore, we further evaluated the performance of Bin2SrcSim under different K values to explore whether the Recall@K of other approaches can surpass that of Bin2SrcSim.

As shown in Fig. 4, Bin2SrcSim demonstrates satisfactory performance across various K values, achieving a 100% recall rate earlier than all comparison approaches in the XA, XO, XC, and XM scenarios. In contrast, CodeLlama-13B and Qwen-14B continue to perform poorly under different K values, with recall rates remaining below 60% in the XA, XO, XC, and XM scenarios even when $K = 50$. The three baselines and GPT-4o exhibit varying Recall@K performance under different K values. Overall, in the XA, XC, and XO scenarios, the recall rates of Gemini, SAFE, Asteria-pro, and GPT-4o stabilize after $K = 40$, all of which exceed 95%. However, in the more complex XM scenario, Gemini and SAFE outperform Asteria-pro and GPT-4o.

In summary, Bin2SrcSim outperforms all comparison baselines and LLMs across cross-compiler, cross-optimization, and cross-architecture scenarios. Even under different K values, Bin2SrcSim maintains the best performance.

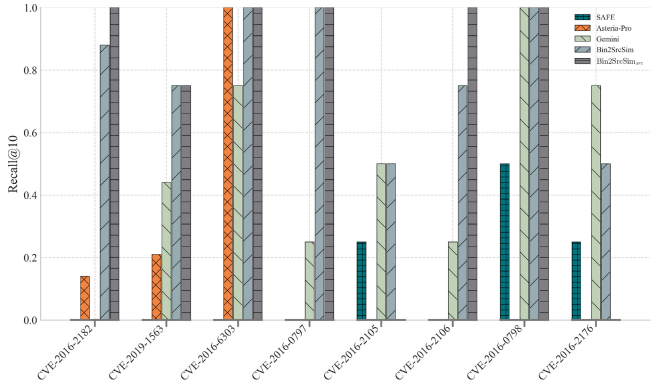

Fig. 5. Recall@10 of real-world vulnerability search. Bin2SrcSim$_{src}$ denotes the use of source code as the query function in vulnerable function search.

C. Vulnerable Function Search

In this section, we evaluate the effectiveness of Bin2SrcSim and baselines in real-world vulnerable function search using Dataset3. Similar to related study [36], we select the libcrypto libraries embedded in two firmware images—Netgear R7000 and TP-Link Deco M4—as the target function pool. We also choose 10 known vulnerable functions as the query functions (covering a total of 8 CVEs), each with four variants (x86, x64, ARM32, MIPS32). Notably, given that Bin2SrcSim supports comparison between binary code and source code, we also extract the source code of each known vulnerable function as the query function. We utilize Recall@10 as the evaluation metric and compare it with three baselines.

Fig. 5 presents the results for the Recall@10 metric for each approach. It is evident that Bin2SrcSim outperforms the three baselines for the majority of CVEs. Specifically, CVE-2016-6303, CVE-2016-0797, and CVE-2016-0798 all achieved a top-10 recall rate of 100%. Taking CVE-2016-0797 as an example, Bin2SrcSim successfully identifies all vulnerable functions in the two firmware images associated with this CVE, whereas Gemini has a Recall@10 value of only 25%, and SAFE and Asteria-Pro fail to recall any vulnerable functions. In contrast, Bin2SrcSim performs slightly less well on CVE-2016-2176, with a Recall@10 value of 50%, lower than Gemini's 75%. Our analysis suggests that this is due to the presence of function inlining and data embedded in

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

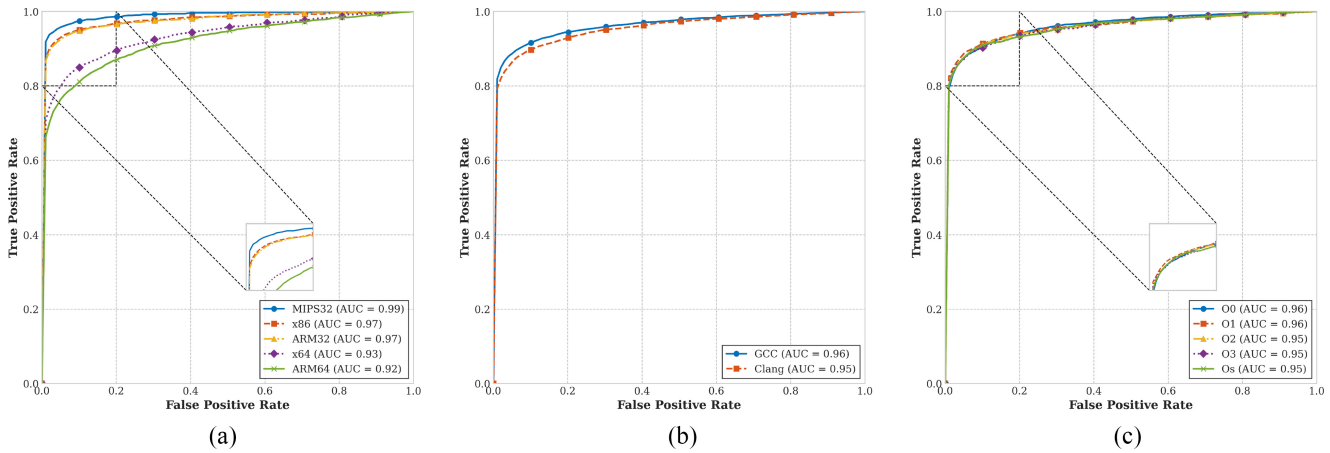

Fig. 6. AUC scores and ROC curves for similarity detection between binary code and source code. (a) Across different architectures. (b) Across different compilers. (c) Across different optimization levels.

assembly code segments in the functions involved, which interfered with similarity detection.

Additionally, Bin2SrcSim demonstrates strong performance in vulnerable function search using source code, achieving a Recall@10 of 100% across five CVEs. For instance, in the case of CVE-2016-2106, $\text{Bin2SrcSim}_{\text{src}}$ successfully identifies all vulnerable functions in the two firmware images associated with this CVE, outperforming both Bin2SrcSim and Gemini. However, $\text{Bin2SrcSim}_{\text{src}}$ underperforms in CVE-2016-2176 and CVE-2016-2105, failing to recall any vulnerable functions. Further analysis indicates that function inlining may be the primary cause of the low recall rate of $\text{Bin2SrcSim}_{\text{src}}$ in these two CVE cases. This compile-time optimization mechanism leads to significant discrepancies between the source code and binary code of the vulnerable functions, thereby disrupting the search results for vulnerable functions. Our analysis suggests that future efforts to identify function inlining and implement nested comparative analysis may effectively mitigate this issue.

Overall, Bin2SrcSim is capable of adapting to real-world vulnerability detection scenarios, especially when the query function involves source code that cannot be compiled into binary code, allowing for effective vulnerable function search.

D. Binary-Source Similarity Detection

In this section, we assess the performance of Bin2SrcSim in detecting similarity between binary code and source code by taking source code as the query function. It should be noted that the baselines do not support similarity detection between binary code and source code. This is because the baseline approaches are designed to detect similarities between pairs of binary code functions. To achieve similarity detection between binary code and source code, an indirect method must be employed, which involves compiling the source code into binary code first. However, when individual source code functions fail to compile successfully due to reasons such as missing dependencies or incompleteness, the baseline approaches are rendered ineffective. In contrast, Bin2SrcSim circumvents this limitation. Bin2SrcSim converts binary code functions into their corresponding source code forms, thereby enabling similarity detection with source code functions without requiring any preprocessing of the source code functions to be detected.

When comparing thesimilarity between binary code and source code, since source code does not involve cross-compiler, cross-optimization level, or cross-architecture, our experimental design involves comparing source code with binary code compiled with different compilers, optimization levels, and for different architectures to evaluate how well Bin2SrcSim can directly compare binary code and source code. The evaluation metric employed is the area under the curve (AUC) of the receiver operating characteristic (ROC) curve, which serves as an aggregate measure of model performance across all possible classification thresholds.

Fig. 6(a) shows the AUC scores for comparisons between binary code and source code across five architectures, such as x86, x64, ARM32, ARM64, and MIPS32. The results indicate that Bin2SrcSim achieves the highest AUC of 0.99 on MIPS32 and the lowest AUC of 0.92 on ARM64, with AUC ranging from 0.93 to 0.97 on x86, x64, and ARM32 architectures. Fig. 6(b) presents the AUC scores for comparisons between binary code and source code under two compilers, such as GCC and Clang. The results show an AUC of 0.96 for GCC and 0.95 for Clang. Fig. 6(c) displays the AUC scores for comparisons between binary code and source code across five optimization levels: O0, O1, O2, O3, and Os. The results demonstrate that Bin2SrcSim maintains robust performance across all optimization levels, with AUC ranging from 0.95 to 0.96.

The analysis reveals that the AUC scores of Bin2SrcSim exhibit a certain degree of fluctuation across different architectures, while remaining relatively stable across different compilers and optimization levels. This indicates that architectural differences have the most pronounced impact on binary code, posing greater challenges for Bin2SrcSim when converting binary code from diverse architectures into source code.

Overall, it can be concluded that Bin2SrcSim effectively supports similarity comparisons between binary code and

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

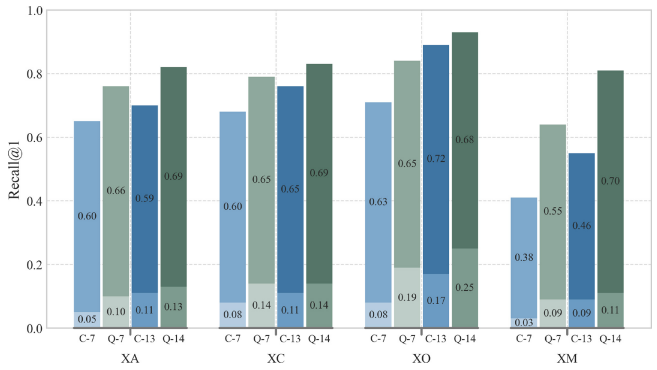

Fig. 7. Recall@1 of different LLMs. C-7 denotes CodeLlama-7B, C-13 denotes CodeLlama-13B, Q-7 denotes Qwen-7B, and Q-14 denotes Qwen-14B. The numbers at the bottom of each bar represent the results of the LLMs without fine-tuning, while the numbers at the top indicate the improvement after fine-tuning.

source code across different compilers, optimization levels, and architectures.

E. Impact of LLM Capabilities

In this section, we select four open-source LLMs: CodeLlama-7B, CodeLlama-13B, Qwen-7B, and Qwen-14B, for fine-tuning within Bin2SrcSim. We aim to evaluate the impact of fine-tuning on these LLMs, as well as the differences between various LLMs and parameters, on BCSD performance using Recall@1 as the evaluation metric on Dataset2.

The experimental results are shown in Fig. 7. It is evident that Bin2SrcSim is effective across all LLMs. Regardless of which LLM is used for fine-tuning, the performance of Bin2SrcSim in BCSD significantly improves after fine-tuning. Without fine-tuning, the Recall@1 values are very low across all scenarios. Specifically, CodeLlama-7B achieves the lowest Recall@1 of 0.03 in the XM scenario, while Qwen-14B achieves the highest Recall@1 of 0.25 in the XO scenario. However, after fine-tuning, all LLMs exhibit substantial performance improvements. For instance, CodeLlama-13B shows an increase in Recall@1 of 59%, 65%, 72%, and 46% in the XA, XC, XO, and XM scenarios, respectively. Similarly, Qwen-14B demonstrates improvements of 69%, 69%, 68%, and 70% in the same scenarios after fine-tuning.

Additionally, it can be observed that when Bin2SrcSim employs Qwen-7B, its performance is comparable to that of CodeLlama-13B. For instance, Qwen-7B outperforms CodeLlama-13B by 6%, 3%, and 9% in the XA, XC, and XM scenarios, respectively, while it lags behind CodeLlama-13B by 5% in the XO scenario. Given that Qwen-7B has nearly half the number of parameters compared to CodeLlama-13B, it can be concluded that Qwen-7B is more efficient in terms of performance.

Further observations reveal that when Bin2SrcSim utilizes Qwen-14B, its performance significantly surpasses that of Qwen-7B. Specifically, Qwen-14B outperforms Qwen-7B by 6%, 4%, 9%, and 17% in the XA, XC, XO, and XM scenarios, respectively. A similar performance improvement is observed when comparing CodeLlama-13B to CodeLlama-7B. This indicates that within the same LLM series, models with larger parameters tend to achieve better performance in BCSD, which aligns with our expectations. This is also the reason we selected Qwen-14B as the LLM for fine-tuning in Bin2SrcSim.

In summary, all LLMs demonstrate effectiveness within Bin2SrcSim, with Qwen-14B achieving the best performance. Moreover, within the same LLM series, LLMs with larger parameters tend to yield better performance in BCSD.

TABLE III
PERFORMANCE OF DIFFERENT REPRESENTATION COMBINATIONS. ASM. DENOTES ASSEMBLY CODE, AND PSD. DENOTES PSEUDOCODE. THE NUMBERS WITH UNDERLINES INDICATE THE SECOND-BEST PERFORMANCE RESULTS

Combinations	Recall@1				MRR			
	XA	XC	XO	XM	XA	XC	XO	XM
Asm.	0.55	0.55	0.73	0.44	0.63	0.61	0.84	0.45
Asm. + CFG	0.80	0.81	0.79	0.63	0.85	0.86	0.83	0.74
Asm. + Psd.	**0.82**	**0.83**	**0.93**	**0.81**	**0.86**	**0.89**	**0.95**	**0.87**
Asm. + CFG + Psd.	0.77	0.81	0.86	0.77	0.84	0.87	0.91	0.83

F. Representation Fine-Tuning Impact

In this section, we investigate the impact on the performance of Bin2SrcSim when different representations are used to fine-tune the LLM. In addition to assembly code and pseudocode, we also opt for CFG, which is widely adopted in other approaches. CFG is constructed from assembly code, which is divided into basic blocks according to program logic. Each basic block is a sequence of consecutive instructions without branches or jumps, having a single entry and exit point. The first instruction typically serves as a branch or jump target, while the last instruction directs the flow to other blocks. Based on these basic blocks, the CFG is thus formed.

Utilizing these three representations, we construct four experimental combinations: 1) assembly code only; 2) assembly code with CFG; 3) assembly code with pseudocode; and 4) assembly code with both CFG and pseudocode. These four combinations were used to fine-tune the LLM in Bin2SrcSim.

As observed from Table III, the combination of assembly code and pseudocode achieves the best performance in BCSD. In the XA, XC, XO, and XM scenarios, this combination's Recall@1 value exceeds that of the second-best combination by 2%, 2%, 7%, and 4%, respectively, and it also attains the highest MRR value. This outcome clearly demonstrates that combining assembly code, which serves as a lower-level representation, with pseudocode, which acts as a higher-level representation, leverages their complementary strengths to enhance detection performance.

Additionally, we observe that CFG plays a positive role in performance enhancement. Compared to the combination using only assembly code, the combination of assembly code and CFG achieved a Recall@1 improvement of 25%, 26%, 6%, and 19% in the XA, XC, XO, and XM scenarios, respectively, with corresponding increases in MRR values except for the XO scenario. This improvement is attributed to the CFG's ability to clearly represent the logical structure of the assembly code.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

However, the incorporation of CFG into the combination of assembly code and pseudocode does not yield the expected benefits. Comparing the combination of assembly code and pseudocode alone with the combination of assembly code, CFG, and pseudocode, it is found that the former achieved higher Recall@1 values by 5%, 2%, 7%, and 4% in the XA, XC, XO, and XM scenarios, respectively. Another interesting finding is that in the XA scenario, the combination of assembly code and CFG has a Recall@1 value 3% higher than the combination of assembly code, CFG, and pseudocode, which is contrary to the results in the other XO, XC, and XM scenarios. Further analysis indicates that for functions with longer assembly code, the generated CFG is also longer. This leads to the combined length of assembly code and CFG exceeding the LLM's maximum token limit, especially when pseudocode is included, thereby degrading performance. Utilizing other LLMs with larger token limits and incorporating CFGs or advanced CFG representations may potentially enhance performance further.

In summary, providing the combination of assembly code and pseudocode to the LLM in Bin2SrcSim for fine-tuning can achieve the best detection performance. CFG is able to contribute to performance enhancement, primarily for functions with shorter assembly code.

V. Conclusion

In this article, we introduce Bin2SrcSim, a novel approach for BCSD that employs an LLM to convert binary code into source code representations. Bin2SrcSim fine-tunes an LLM at the function level to transform assembly code and pseudocode into source code. Benefiting from this, the LLM can supplement and enhance the high-level semantic and structural information lost during binary code compilation. Consequently, the similarity between any two binary code functions can be assessed by calculating the cosine similarity and Jaccard similarity of the transformed source code. Experimental results demonstrate that Bin2SrcSim outperforms all baseline approaches. Moreover, Bin2SrcSim supports similarity detection between binary code and source code, thereby expanding the scope of BCSD applications.

Future work will explore more complex scenarios, such as detection effectiveness in the presence of inline functions, and investigate the BCSD approach that does not rely on disassembly tools, leveraging the capabilities of LLMs.

References

[1] "2023 IoT security report." NETGEAR. 2023. Accessed: Aug. 25, 2024. [Online]. Available: https://www.netgear.com/hub/network/2023-iot-security-report

[2] A. Cui, M. Costello, and S. J. Stolfo, "When firmware modifications attack: A case study of embedded exploitation," in *Proc. Netw. Distrib. System Security Symp.*, 2013, pp. 1–13.

[3] S. Eschweiler, K. Yakdan, and E. Gerhards-Padilla, "DiscovRE: Efficient cross-architecture identification of bugs in binary code," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2016, pp. 1–15.

[4] A. Sæbjørnsen, J. Willcock, T. Panas, D. Quinlan, and Z. Su, "Detecting code clones in binary executables," in *Proc. 18th Int. Symp. Softw. Test. Anal.*, 2009, pp. 117–128.

[5] F. Zuo, X. Li, P. Young, L. Luo, Q. Zeng, and Z. Zhang, "Neural machine translation inspired binary code similarity comparison beyond function pairs," in *Proc. Netw. Distrib. System Security Symp.*, 2019, pp. 1–15.

[6] S. H. H. Ding, B. C. M. Fung, and P. Charland, "Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *Proc. IEEE Symp. Security Privacy (SP)*, 2019, pp. 472–489.

[7] D. Tian, X. Jia, R. Ma, S. Liu, W. Liu, and C. Hu, "BinDeep: A deep learning approach to binary code similarity detection," *Expert Syst. Appl.*, vol. 168, Apr. 2021, Art. no. 114348.

[8] Z. Xu, B. Chen, M. Chandramohan, Y. Liu, and F. Song, "SPAIN: Security patch analysis for binaries towards understanding the pain and pills," in *Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. (ICSE)*, 2017, pp. 462–472.

[9] H. He et al., "Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection," in *Proc. 33rd USENIX Security Symp.*, 2024, pp. 1759–1776.

[10] L. Massarelli, G. A. Di Luna, F. Petroni, R. Baldoni, and L. Querzoni, "SAFE: Self-attentive function embeddings for binary similarity," in *Proc. Int. Conf. Detect. Intrusions Malware, Vulnerabil. Assess.*, 2019, pp. 309–329.

[11] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "TREX: Learning execution semantics from micro-traces for binary similarity," 2020, *arXiv:2012.08680*.

[12] Q. Feng, R. Zhou, C. Xu, Y. Cheng, B. Testa, and H. Yin, "Scalable graph-based bug search for firmware images," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 480–491.

[13] D. Peng, S. Zheng, Y. Li, G. Ke, D. He, and T.-Y. Liu, "How could neural networks understand programs?" in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 8476–8486.

[14] Z. Luo et al., "VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search," in *Proc. Netw. Distrib. System Security Symp.*, 2023, pp. 1–18.

[15] W. Zhang, Z. Xu, Y. Xiao, and Y. Xue, "Unleashing the power of pseudocode for binary code similarity analysis," *Cybersecurity*, vol. 5, no. 1, p. 23, 2022.

[16] A. Hurst et al., "GPt-4o system card," 2024, *arXiv:2410.21276*.

[17] B. Roziere et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.

[18] B. Hui et al., "Qwen2.5-coder technical report," 2024, *arXiv:2409.12186*.

[19] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 363–376.

[20] S. Yang et al., "Asteria-Pro: Enhancing deep learning-based binary code similarity detection by incorporating domain knowledge," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 1, pp. 1–40, 2023.

[21] C. Qu, R. Zhou, K. Liu, Z. Yan, and H. Qu, "BSGAT: A graph attention network for binary code similarity detection," in *Proc. IEEE 28th Pacific Rim Int. Symp. Dependable Comput. (PRDC)*, 2023, pp. 161–167.

[22] Y. Duan, X. Li, J. Wang, and H. Yin, "DeepBinDiff: Learning program-wide code representations for binary diffing," in *Proc. Netw. Distrib. System Security Symp.*, 2020, pp. 1–16.

[23] Z. Luo, P. Wang, W. Xie, X. Zhou, and B. Wang, "IoTSim: Internet of Things-oriented binary code similarity detection with multiple block relations," *Sensors*, vol. 23, no. 18, p. 7789, 2023.

[24] N. Cui, L. Chen, and G. Shi, "BVSNO: Binary code vulnerability detection based on slice semantic and node order," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, 2023, pp. 373–379.

[25] H. Wang et al., "jTrans: Jump-aware transformer for binary code similarity detection," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, 2022, pp. 1–13.

[26] G. Liu, X. Zhou, J. Pang, F. Yue, W. Liu, and J. Wang, "Codeformer: A GNN-nested transformer model for binary code similarity detection," *Electronics*, vol. 12, no. 7, p. 1722, 2023.

[27] Y. Gu, H. Shu, F. Kang, and F. Hu, "UniASM: Binary code similarity detection without fine-tuning," *Neurocomputing*, vol. 630, May 2025, Art. no. 129646.

[28] J. Yang, C. Fu, X.-Y. Liu, H. Yin, and P. Zhou, "Codee: A tensor embedding scheme for binary code search," *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2224–2244, Jul. 2022.

[29] J. Xing, S. Luo, L. Pan, J. Hao, Y. Guan, and Z. Wu, "HGE-BVHD: Heterogeneous graph embedding scheme of complex structure functions for binary vulnerability homology discrimination," *Expert Syst. Appl.*, vol. 238, Mar. 2024, Art. no. 121835.

[30] J. Křoustek, P. Matula, and P. Zemek, "RetDec: An open-source machine-code decompiler," in *Proc. Botconf*, 2017, pp. 1–36.

[31] A. Albalak et al., "A survey on data selection for language models," 2024, *arXiv:2402.16827*.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.

[32] A. Aghajanyan, S. Gupta, and L. Zettlemoyer, "Intrinsic dimensionality explains the effectiveness of language model fine-tuning," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguist. 11th Int. Joint Conf. Nat. Lang. Process.*, 2021, pp. 7319–7328.
[33] N. Ding et al., "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nat. Mach. Intell.*, vol. 5, no. 3, pp. 220–235, 2023.
[34] E. J. Hu et al., "LoRa: Low-rank adaptation of large language models," in *Proc. 10th Int. Conf. Learn. Represent.*, 2022, pp. 1–36.
[35] Y. Zheng, R. Zhang, J. Zhang, Y. Ye, and Z. Luo, "LlamaFactory: Unified efficient fine-tuning of 100+ language models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguist.*, 2024, pp. 1–13.
[36] A. Marcelli, M. Graziano, X. Ugarte-Pedrero, Y. Fratantonio, M. Mansouri, and D. Balzarotti, "How machine learning is solving the binary function similarity problem," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 2099–2116.

Bangrui Wan received the M.S. degree in computer software and theory from Southwest University, Chongqing, China, in 2007.

He accumulated 11 years of development experience in the software industry, led the Research and Development of two carrier-grade Internet of Things platforms. His current main research areas focus on code security assurance, automated code generation, and software testing.

Shiyu Wang is currently pursuing the M.S. degree in software engineering with Chongqing University of Posts and Telecommunications, Chongqing, China.

His primary research focuses on binary code similarity detection and automated code generation.

Zheng Wei received the B.S. degree in software engineering from Chongqing University of Posts and Telecommunications, Chongqing, China, in 2018, where he is currently pursuing the M.S. degree.

His current research interests include binary code similarity detection and code clone detection.

Jiangping Huang received the Doctoral degree in computer software and theory from Wuhan University, Wuhan, China, in 2017.

He was a Postdoctoral Researcher with Nanyang Technological University, Singapore. In recent years, he has participated in one National Natural Science Foundation Key Project, served as a Core Researcher in three NSFC General Projects and one National Social Science Foundation Project, and independently led six provincial or ministerial-level projects and three university-level projects.

Chunqiang Hu (Member, IEEE) received the B.S. degree in computer science and technology from Southwest University, Chongqing, China, in 2006, and the M.S. and Ph.D. degrees in computer science and technology from Chongqing University, Chongqing, in 2009 and 2013, respectively, and the second Ph.D. degree in computer science from The George Washington University, Washington, DC, USA, in 2016.

From 2011 to 2012, he was a Visiting Scholar with The George Washington University. He is currently a Professor with the School of Big Data and Software Engineering, Chongqing University. His research interests include blockchain, privacy computing, data-driven security, applied cryptography, and algorithm design and analysis.

Dr. Hu was honored with the Hundred-Talent Program by Chongqing University. He is a Senior Member of the China Computer Federation.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on February 23,2026 at 05:36:08 UTC from IEEE Xplore. Restrictions apply.